Lab 4: Vectors, Open Source — Chesapeake Stations

Overview

This lab applies the GeoPandas vector toolkit to the real question: **how well does the Chesapeake Bay Program's tidal water-quality monitoring network cover the seven-state watershed it serves?**

Two files are provided: 1. `cbw_stations.geojson` holds 166 tidal monitoring stations 2. `cbw_counties.geojson` holds 205 counties and independent cities across Virginia, Maryland, Pennsylvania, New York, West Virginia, Delaware, and the District of Columbia.

There are four tasks, each with a few sub-steps

- **Task A** inventories both files and reprojects them to a shared, appropriate coordinate reference system
- **Task B** joins stations to counties by location to identify which counties the network serves (and which it does not)
- **Task C** refines the definition of "unserved" by buffering the stations and testing whether a county without a station nonetheless lies close to one
- **Task D** is a short written assessment of what would need checking before this analysis could be trusted on a larger dataset

Download the template notebook, `lab04_template.ipynb`, before you start. It contains one markdown header and one seeded code (or markdown) cell per task, in order. Complete your work inside the template and submit the finished file.

Setup

Every Part 2 notebook opens with the same installation cell:

```
# Run this cell first (takes ~1 minute). Colab already includes geopandas.
%pip install -q libpysal esda splot mapclassify contextily
```

Data. Both files load by URL through the `DATA_BASE_P2` constant:

```
DATA_BASE_P2 = "https://raw.githubusercontent.com/pjbitterman/KSU_x9076/main/data/part2"

# Local path fallback (uncomment if running outside Colab without internet access):
# DATA_BASE_P2 = "../data/part2"
```

- `cbw_stations.geojson`: 166 tidal water-quality monitoring stations (points), with `station_id`, `latitude`, `longitude`, `station_type` (`main` or `trib`), `tidal_status`, `state`, `waterbody`, `cbw_segment`, `min_year`, and `max_year`. Some rows carry null values in the metadata columns. Task A asks you to quantify that.
- `cbw_counties.geojson`: 205 counties and independent cities (polygons), with `state_fp`, `county_fp`, `geoid`, `name`, `name_long`, and `state`.

Both files were used in the week 9 sessions. Nothing new needs to be downloaded.

Task A: Inventory and reprojection (20 points)

Build a documented inventory of both files, then place them on a coordinate reference system suitable for the remaining tasks

1. Load `cbw_stations.geojson` into `stations` and `cbw_counties.geojson` into `counties`, both from `DATA_BASE_P2`
2. For each, print the row count, the full column list (`.columns.tolist()`), and the CRS (`.crs`)
3. Report how many rows in `stations` are missing a value in each column (`.isna().sum()`). No handling of the missing values is required at this stage. The requirement is to know how many there are and which columns they occupy
4. Reproject **both** GeoDataFrames into a single projected CRS in meters that is appropriate for the region: `EPSG:26918` (NAD83 / UTM zone 18N). Store the results as `stations_p` and `counties_p`. Print `.total_bounds` for each before and after the reprojection to confirm that the coordinate scale changed (values of a few degrees before, six- to seven-digit values in meters after)

Use `stations_p` and `counties_p`, not the geographic originals, for every task that follows. Buffering, distance, and area require linear units, and a shared projected CRS lets `sjoin` compare the two layers without a coordinate-system mismatch.

> **Why reproject both files?** `sjoin` will run on two layers whose CRSs differ and issue only a warning. In the week 9 session this produced no visible error because EPSG:4326 and EPSG:4269 agree closely at this scale, but that is a property of those two systems, not a guarantee. Reprojecting both inputs into one CRS before the join removes the dependence on that coincidence.

Task B: Spatial join and per-county coverage (25 points)

Worked example: the join-and-count pattern. The structure of the task, demonstrated on two small illustrative GeoDataFrames (not `stations` and `counties`):

```
import geopandas as gpd

# points_gdf: a handful of sensor locations; zones_gdf: a handful of zone polygons
joined = gpd.sjoin(points_gdf, zones_gdf[["zone_id", "zone_name", "geometry"]],
                   how="left", predicate="within")
print(joined.shape)  # may be >= len(points_gdf): a point on a shared edge can match twice

matched = joined[joined["zone_name"].notna()]
per_zone = (
    matched.groupby(["zone_id", "zone_name"])
    .size()
    .reset_index(name="point_count")
    .sort_values("point_count", ascending=False)
)
print(per_zone)
```

Three steps: 1. join by location (`sjoin` with `predicate="within"`), 2. keep only the rows that matched (`.notna()` on a column that exists only on the right-hand side), 3. then `groupby` and `.size()` to convert one row per point into one row per zone with a count. The task below follows this structure exactly.

The join. Using `stations_p` and `counties_p` from Task A:

1. Run `gpd.sjoin(stations_p, counties_p[["geoid", "name", "state", "geometry"]], how="left", predicate="within")`. Both

inputs carry a `state` column, so `sjoin` suffixes the collision: the county's state is returned as `state_right` and the station's as `state_left`
2. Compare the output's row count with `len(stations_p)`. If they differ, determine why. Report each case in a sentence: how many rows, and which stations.
3. Restrict to the rows that matched a county and build a per-county station-count table with `geoid`, `name`, `state_right` (or rename it to `state`), and `station_count`, sorted with the best-served counties first
4. Report the **five best-served counties** by `station_count`, as a small table or a few printed lines
5. Report **five counties with zero stations**. A `groupby` on the joined table will not produce a row for a county that matched nothing, so compare your per-county table's county list against the full `counties_p` list to find them (an `.isin()` check against the served counties' `geoid` values is one approach). ***In a sentence,*** describe the geographic pattern in where the zero-station counties cluster. Far more than five counties have zero stations, so there is no single correct set of five; report five real ones and the pattern that connects them.

Task C: Buffer coverage (18 points)

Task B asked a strict question: does a station lie inside this county's polygon? A county with no station inside it is not necessarily far from monitoring. It may border a well-served county and have a station a few kilometers past the line. This task asks a more useful version.

Why this variant. `cbw_stations.geojson` and `cbw_counties.geojson` are the only two files in this lab. There is no county-seat point layer or shoreline line layer to buffer against, so Task C buffers the stations themselves and asks which of Task B's zero-station counties fall within that buffer.

1. Buffer `stations_p` by a distance of your choice, in meters. **10,000 m (10 km) is a reasonable default.** If you choose a different value, justify it in a sentence; a materially smaller or larger radius changes which counties count as "near" a station, so the choice must be deliberate.
2. For every county in `counties_p`, determine whether its polygon intersects **any** station's buffer, not only whether a station lies inside the county.
3. Using Task B's per-county station-count table, classify every county into exactly one of three categories:
   - **served**: one or more stations inside the county (Task B's non-zero counties)
   - **near**: zero stations inside, but the county intersects at least one station's buffer

- **unserved**: zero stations inside, and the county intersects no station's buffer

4. Report the count of counties in each category. The three counts must sum to 205.
5. Map the result: a choropleth of `counties_p` colored by category (three distinct colors, with a legend), with the station points plotted on top. The title must state the buffer distance you used.

Task D: What would break? (8 points)

In 200 to 300 words, answer the following. **If a colleague reran this exact analysis against the Chesapeake Bay Program's full, current, multi-decade station roster instead of this lab's `cbw_stations.geojson`, what would you expect to change, and what in your Task A through C code might produce a wrong answer silently rather than fail with an error?**

Ground the answer in something you observed in this lab rather than a generic list of risks. Starting points, if useful (choose what you noticed; do not restate all three):

- Task A's null-count check describes how complete this roster's metadata is. Would a larger, current roster be as complete, more, or less, and does anything in your Task B or C code assume the metadata you have now?
- `cbw_stations.geojson` loads from a URL, not a versioned local file. What happens to Task B's five best-served counties if the file at that URL changes between your run today and a colleague's run next month?
- A larger roster likely means more stations and a wider date range per station (`min_year` and `max_year` are already columns in this file). Does anything in Task B's join or Task C's buffer distance implicitly assume the current station count or spatial spread?

Deliverables and submission

Submit through Canvas:

1. Your completed template notebook (`lab04_template.ipynb`), containing Task A's inventory and reprojection, Task B's join and per-county coverage tables, Task C's buffer classification and map, and Task D's prose, in one file that runs top to bottom.

Point breakdown

Component	Points
Task A: Inventory and reprojection	20
Task B: Spatial join and per-county coverage	25
Task C: Buffer coverage	18
Task D: What would break?	8
Style and naming	4
Total	**75**

"Style and naming" covers `snake_case` variable names, f-strings for all formatted output, and derived (not hand-typed) values wherever the lab asks for something computed, the same conventions as Part 1's labs.

Late work follows the course late policy